REVISION CHEAT SHEET: OBJECT RELATIONSHIPS

Composition, Association, and Object Relationships in C#

Interview-ready notes · examples · C# · diagrams

Topic 16 of 40

Object-oriented applications are not built from isolated classes. Objects collaborate, reference one another, own other objects, receive dependencies, and participate in larger domain models.

The key question is not only:

EXAMPLE / FLOW

```
What does this class contain?
```

It is also:

EXAMPLE / FLOW

```
How is this object related to other objects?
Who owns whom?
Who controls lifetime?
Who is allowed to mutate the relationship?
Is this IS-A or HAS-A?
```

In the Fundraising Platform, relationships include:

EXAMPLE / FLOW

```
Constituent -> Gifts
Gift -> Campaign
Gift -> GiftAllocations
GiftAllocation -> Fund
Constituent <-> Address through ConstituentAddress
GiftService -> IGiftRepository
GiftService -> IGiftReceiptSender
```

These relationships do not all mean the same thing.

High-Level Mental Model

EXAMPLE / FLOW

```
IS-A
    -> inheritance

HAS-A
    -> composition / aggregation / association

USES-A
    -> dependency / collaboration
```

Choosing the correct relationship affects coupling, testability, lifetime, persistence modeling, and maintainability.

Topic Coverage

This topic covers:

- association;
- aggregation;
- composition;
- dependency relationships;
- object collaboration;
- HAS-A relationships;
- IS-A relationships;
- inheritance vs composition;
- composition over inheritance;
- ownership;
- lifetime dependency;
- one-to-one relationships;
- one-to-many relationships;
- many-to-many relationships;
- link/association objects;
- constructor injection;
- service composition;
- collection ownership;
- bidirectional relationships;
- avoiding cyclic object graphs;
- encapsulating relationship changes;
- domain invariants across relationships;
- ASP.NET Core dependency injection;
- persistence/EF Core awareness;
- Fundraising Platform examples;
- performance considerations;
- security considerations;
- common mistakes;
- interview questions;
- practical exercises.

Why Object Relationships Matter

A class can be individually well-designed and still participate in a poor object model.

For example:

```C#
public sealed class Gift
{
    public Campaign Campaign { get; set; }
    public Fund Fund { get; set; }
    public Constituent Constituent { get; set; }
    public AppUser CreatedBy { get; set; }
}
```

This exposes several relationships, but important questions remain:

EXAMPLE / FLOW

```
Can Campaign be null?
Can a Gift move to another Constituent?
Can a Gift have multiple Funds?
Who updates both sides of a relationship?
Who owns GiftAllocation lifetime?
Can callers replace relationship objects freely?
```


Relationship design answers those questions explicitly.

Part 1 - Association

Definition

Association is a general relationship in which one object knows about, references, or interacts with another object.

It is the broadest relationship category in this topic.

Example:

C#

```
public sealed class Gift
{
    public Campaign? Campaign { get; private set; }
}
```

A `Gift` is associated with a `Campaign`.

The campaign can exist independently of that gift.

Association Does Not Necessarily Mean Ownership

EXAMPLE / FLOW

```
Gift -> Campaign
```

does not automatically mean:

EXAMPLE / FLOW

```
Gift owns Campaign
```

Deleting a gift should not normally delete the campaign.

The relationship is about connection, not necessarily lifecycle control.

One-Way Association

C#

```
public sealed class Gift
{
    public Campaign? Campaign { get; private set; }
}
```

The gift knows its campaign.

The campaign might not expose a collection of gifts in the in-memory model.

This is a **unidirectional association**.

Two-Way Association

```C#
public sealed class Campaign
{
    private readonly List<Gift> _gifts = [];
    public IReadOnlyList<Gift> Gifts => _gifts;
}
```

and:

```C#
public sealed class Gift
{
    public Campaign? Campaign { get; private set; }
}
```

Now both objects can navigate the relationship.

This is a **bidirectional association**.

Bidirectional Relationships Need Discipline

If callers can independently do:

```C#
campaign.Gifts.Add(gift);
gift.Campaign = anotherCampaign;
```

the graph can become inconsistent.

Prefer relationship methods that update state intentionally.

Encapsulated Association

```C#
public sealed class Gift
{
    public Campaign? Campaign { get; private set; }

    public void AssignCampaign(Campaign campaign)
    {
        ArgumentNullException.ThrowIfNull(campaign);
        Campaign = campaign;
    }
}
```

Now the relationship changes through domain behavior rather than unrestricted setters.

Part 2 - Aggregation

Definition

Aggregation is commonly used to describe a whole-part relationship where the part can exist independently of the whole.

Example mental model:

EXAMPLE / FLOW

```
Campaign
    -> has Gifts

but Gift can still exist conceptually
without being owned for its entire lifetime by Campaign
```

In many C# codebases, aggregation is not represented by a special language keyword. It is a design interpretation of references and lifetime.

Aggregation Example

C#

```
public sealed class Campaign
{
    private readonly List<Gift> _gifts = [];

    public IReadOnlyList<Gift> Gifts => _gifts;

    public void AddGift(Gift gift)
    {
        ArgumentNullException.ThrowIfNull(gift);
        _gifts.Add(gift);
    }
}
```

The campaign groups gifts, but a gift has its own identity and lifecycle.

Aggregation Mental Model

EXAMPLE / FLOW

```
Whole
    -> references parts

Parts
    -> can exist independently

Destroy whole
    -> parts do not necessarily cease to exist
```

Part 3 - Composition

Definition

Composition is a strong HAS-A relationship in which an object is built from collaborating parts and often controls or strongly defines their lifetime or use.

Example:

EXAMPLE / FLOW

```
Gift
    HAS-A
GiftAllocation collection
```

If `GiftAllocation` is meaningful only as part of a particular gift's allocation structure, the gift may be the natural owner of that relationship.

Composition Example

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation> _allocations = [];

    public IReadOnlyList<GiftAllocation> Allocations => _allocations;

    public void AllocateTo(Fund fund, decimal amount)
    {
        ArgumentNullException.ThrowIfNull(fund);

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(nameof(amount));
        }

        _allocations.Add(new GiftAllocation(fund, amount));
    }
}
```

The caller does not directly construct and attach arbitrary allocation state through a public mutable list.

The gift coordinates the composition.

Composition Mental Model

EXAMPLE / FLOW

```
Owner object
    -> contains / receives collaborators
    -> coordinates behavior
    -> protects invariants
```

Composition is not only about physical containment. It is about building behavior from other objects instead of inheriting it.

Composition for Behavior

A service may compose several dependencies:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;
    private readonly IGiftReceiptSender _receiptSender;
    private readonly IClock _clock;

    public GiftService(
        IGiftRepository repository,
        IGiftReceiptSender receiptSender,
        IClock clock)
    {
        _repository = repository;
        _receiptSender = receiptSender;
        _clock = clock;
    }
}
```

`GiftService` **has** these collaborators.

It does not inherit from them.

HAS-A Relationship

EXAMPLE / FLOW

```
GiftService HAS-A IGiftRepository
GiftService HAS-A IGiftReceiptSender
Gift HAS-A collection of GiftAllocation
Campaign HAS-A collection of Gifts
```

HAS-A usually points toward composition or association rather than inheritance.

IS-A Relationship

EXAMPLE / FLOW

```
GiftTransaction IS-A FinancialTransaction
RefundTransaction IS-A FinancialTransaction
```

IS-A is the conceptual test for inheritance.

Do not use inheritance when the relationship is really HAS-A.

Bad Inheritance for a HAS-A Relationship

Bad:

C#

```
public class GiftService : SqlGiftRepository
{
}
```

A gift service is not a SQL repository.

The service merely **uses** a repository.

Better:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;

    public GiftService(IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

Inheritance vs Composition

Inheritance	Composition
IS-A relationship	HAS-A relationship
Behavior comes from base type	Behavior comes from collaborators
Strong compile-time coupling to base	Collaborators can often be replaced
One direct base class	Many collaborators possible

Inheritance	Composition
Runtime specialization through override	Delegation to contained dependency
Base changes can affect derived types	Component can often change behind interface
Good for true subtype hierarchies	Good for assembling behavior

Composition Over Inheritance

The phrase **prefer composition over inheritance** does not mean inheritance is bad.

It means:

EXAMPLE / FLOW

```
Do not inherit merely to reuse code.
```

If a relationship is not genuinely IS-A, composition is usually safer and more flexible.

Why Composition Is Often Preferred

Composition can reduce several inheritance problems:

EXAMPLE / FLOW

```
Deep hierarchy coupling
Fragile base classes
Too many protected members
Unwanted inherited API surface
Rigid one-base-class restriction
Behavior fixed by class hierarchy
```

Composition allows behavior to be assembled from focused collaborators.

Delegation

Composition often uses delegation:

C#

```
public sealed class GiftService
{
    private readonly IGiftReceiptSender _receiptSender;

    public GiftService(IGiftReceiptSender receiptSender)
    {
        _receiptSender = receiptSender;
    }

    public Task SendReceiptAsync(Gift gift, CancellationToken cancellationToken)
    {
        return _receiptSender.SendAsync(gift, cancellationToken);
    }
}
```

The service delegates receipt behavior to the collaborator.

Dependency Relationship

Definition

A **dependency** exists when one component needs another component to perform its work.

Examples:

EXAMPLE / FLOW

```
GiftService uses IGiftRepository
Controller uses IGiftService
Repository uses DbContext
ReceiptSender uses email client
```

A dependency may be stored as a field, passed temporarily as a method argument, or created internally.

Constructor Dependency

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;

    public GiftService(IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

This is an explicit, stable dependency.

Method Dependency

C#

```
public void Export(Gift gift, IExportWriter writer)
{
    writer.Write(gift);
}
```

The object does not necessarily retain the dependency after the call.

Hidden Dependency

Bad:

C#

```
public sealed class GiftService
{
    public void Save(Gift gift)
    {
        var repository = new SqlGiftRepository();
        repository.Save(gift);
    }
}
```

The dependency is hidden inside implementation code.

This makes testing and replacement harder.

Explicit Dependency

Better:

```C#
public sealed class GiftService
{
    private readonly IGiftRepository _repository;

    public GiftService(IGiftRepository repository)
    {
        _repository = repository;
    }
}
```

The constructor communicates:

```EXAMPLE / FLOW
GiftService cannot do its job without IGiftRepository
```

Object Collaboration

Objects collaborate when one asks another to perform part of a larger operation.

Example:

```EXAMPLE / FLOW
GiftsController
    -> GiftService
        -> Gift domain object
        -> IGiftRepository
        -> IGiftReceiptSender
        -> IOutboxWriter
```

Each object should have a focused responsibility.

Collaboration Flow

```EXAMPLE / FLOW
HTTP request
    -> Controller
    -> Application service
    -> Domain object
    -> Repository
    -> Database
    -> Receipt sender / outbox
```

Good collaboration makes responsibilities visible instead of putting every operation into one giant class.

One-to-One Relationship

Example:

```EXAMPLE / FLOW
Constituent
    -> PrimaryProfile
```

In code:

```C#
public sealed class Constituent
{
```

```
    public ConstituentProfile Profile { get; private set; }
}
```


The actual ownership semantics depend on the domain.

One-to-one cardinality alone does not tell you whether the relationship is composition or loose association.

One-to-Many Relationship

Fundraising example:

EXAMPLE / FLOW

```
Constituent 1 -> many Gifts
Campaign 1 -> many Gifts
Gift 1 -> many GiftAllocations
```

Collection modeling:

C#

```
private readonly List<Gift> _gifts = [];
public IReadOnlyList<Gift> Gifts => _gifts;
```

Expose controlled mutation through methods when invariants matter.

Many-to-Many Relationship

Fundraising example:

EXAMPLE / FLOW

```
Constituent many <-> many Address
through ConstituentAddress
```

and:

EXAMPLE / FLOW

```
Gift many <-> many Fund
through GiftAllocation
```

In domain-rich systems, the join/link object often has meaning of its own.

Association Object / Link Entity

`GiftAllocation` is more than a pure connection because it can contain data such as:

EXAMPLE / FLOW

```
GiftId
FundId
Amount
Percentage
CreatedAt
```

That makes it an association object/entity.

EXAMPLE / FLOW

```
Gift
    -> GiftAllocation
        -> Fund
```

Why Link Objects Matter

A direct collection like:

```C#
List<Fund>
```

cannot naturally store allocation-specific information such as amount.

A relationship object can.

Fundraising Relationship Map

EXAMPLE / FLOW

```
Tenant
  -> AppUsers
  -> Constituents
  -> Campaigns
  -> Funds

Constituent
  -> Gifts
  <-> Addresses through ConstituentAddress

Gift
  -> Campaign
  -> GiftAllocations

GiftAllocation
  -> Fund
```

Different edges represent different business semantics.

Constituent to Gift

EXAMPLE / FLOW

```
Constituent 1 -> many Gift
```

A gift belongs to one constituent in the current model.

The constituent and gift are both independent domain entities with identities.

This is usually an association rather than simple object composition in the sense of lifetime destruction.

Gift to Campaign

EXAMPLE / FLOW

```
Campaign 1 -> many Gift
Gift -> optional Campaign
```

A campaign can exist without any gifts.

A gift may exist without a campaign depending on the business rules.

This strongly suggests association/aggregation semantics rather than strict lifetime composition.

Gift to GiftAllocation

EXAMPLE / FLOW

```
Gift 1 -> many GiftAllocation
```

If allocations only make sense within a gift and are managed through the gift, this is a strong composition candidate in the domain model.

The gift can protect rules such as:

EXAMPLE / FLOW

```
Allocation amounts must be positive
Allocation total must not exceed gift amount
Fund must be valid for tenant
Duplicate allocation rules
```

GiftAllocation to Fund

EXAMPLE / FLOW

```
GiftAllocation -> Fund
```

The allocation references a fund.

The allocation does not own the fund's lifecycle.

Deleting an allocation should not delete the fund.

Encapsulate Collections

Bad:

C#

```
public List<GiftAllocation> Allocations { get; set; } = [];
```

Any caller can:

C#

```
Gift.Allocations.Clear();
```

or add invalid state.

Better:

C#

```
private readonly List<GiftAllocation> _allocations = [];
public IReadOnlyList<GiftAllocation> Allocations => _allocations;
```

Mutation goes through methods.

Relationship Invariants

Example:

C#

```
public void AllocateTo(Fund fund, decimal amount)
{
    ArgumentNullException.ThrowIfNull(fund);
```

```
    if (amount <= 0m)
    {
        throw new ArgumentOutOfRangeException(nameof(amount));
    }

    decimal allocated = _allocations.Sum(x => x.Amount);

    if (allocated + amount > TotalAmount)
    {
        throw new InvalidOperationException("Allocations cannot exceed gift amount.");
    }

    _allocations.Add(new GiftAllocation(fund, amount));
}
```

The owner protects the relationship invariant.

Ownership

Ask:

EXAMPLE / FLOW

```
Who is allowed to create the part?
Who is allowed to remove it?
Can the part exist independently?
Who protects invariants?
Who controls persistence changes?
```

Those questions are more useful than merely labeling everything "composition."

Lifetime Dependency

Strong composition often implies:

EXAMPLE / FLOW

```
Owner lifetime
    -> controls part lifetime
```

But in real enterprise systems, database lifecycle, domain lifecycle, and object-in-memory lifetime do not always map perfectly.

Use lifecycle semantics deliberately rather than mechanically applying UML labels.

Service Composition

Application services are often composed from interfaces:

C#

```
public sealed class GiftService
{
    private readonly IGiftRepository _repository;
    private readonly IGiftReceiptSender _receiptSender;
    private readonly IOutboxWriter _outboxWriter;

    public GiftService(
        IGiftRepository repository,
        IGiftReceiptSender receiptSender,
        IOutboxWriter outboxWriter)
    {
        _repository = repository;
        _receiptSender = receiptSender;
```

```
        _outboxWriter = outboxWriter;
    }
}
```


This is composition at the service level.

Composition and Dependency Injection

DI containers assemble object graphs:

EXAMPLE / FLOW

```
GiftsController
    -> IGiftService
        -> GiftService
            -> IGiftRepository
            -> IGiftReceiptSender
            -> IOutboxWriter
```

The DI container creates and supplies collaborators according to configured lifetimes.

DI Is Object Composition Infrastructure

Dependency injection is not the same as composition, but it is a common way to **build composed object graphs**.

The service still has a HAS-A relationship with its dependencies.

The container simply wires them together.

Avoid Service Locator

Bad:

C#

```
public sealed class GiftService
{
    private readonly IServiceProvider _services;

    public GiftService(IServiceProvider services)
    {
        _services = services;
    }

    public void Process()
    {
        var repository = _services.GetRequiredService<IGiftRepository>();
    }
}
```

Dependencies are hidden behind a generic service provider.

Prefer explicit constructor parameters for required collaborators.

Composition and Interfaces

Composition becomes especially flexible when collaborators are abstractions:

EXAMPLE / FLOW

```
GiftService
    HAS-A
IGiftRepository
```

instead of:

EXAMPLE / FLOW

```
GiftService
    HAS-A
SqlGiftRepository
```

This lets infrastructure vary without changing the service contract.

Composition and Polymorphism

Composition and polymorphism work together:

EXAMPLE / FLOW

```
GiftService
    -> IPaymentProcessor
        -> CardPaymentProcessor
        -> AchPaymentProcessor
```

Composition says:

EXAMPLE / FLOW

```
GiftService HAS-A processor
```

Polymorphism says:

EXAMPLE / FLOW

```
Processor implementation can vary
```

Composition and Encapsulation

Composition without encapsulation can still be dangerous.

Bad:

C#

```
public List<GiftAllocation> Allocations { get; set; }
```

Better:

C#

```
private readonly List<GiftAllocation> _allocations = [];
public IReadOnlyList<GiftAllocation> Allocations => _allocations;
```

The owner exposes only the relationship operations callers need.

Composition and Immutability

Some composed objects can be immutable:

C#

```
public sealed record PostalAddress(
    string Line1,
```

```
    string City,
    string State,
    string PostalCode);
```

An entity might hold an address value while replacement creates a new address value rather than mutating shared state.

Choose semantics based on whether the nested concept is an entity, value object, or shared reference.

Cyclic Object Graphs

Bidirectional relationships can create cycles:

EXAMPLE / FLOW

```
Campaign
    -> Gifts
        -> Campaign
```

This can complicate:

EXAMPLE / FLOW

```
Serialization
Debug printing
Equality
Traversal
Memory retention
Mapping
```

Do not make every relationship bidirectional automatically.

Serialization Cycles

A JSON serializer may encounter:

EXAMPLE / FLOW

```
Campaign -> Gift -> Campaign -> Gift -> ...
```

Use DTOs/projections for API responses rather than serializing rich persistence/domain graphs directly.

Equality and Relationships

Be careful including mutable navigation collections in equality.

Entity equality should usually focus on identity semantics.

Value-object equality can be structural when that matches the concept.

Do not create record-style value equality across large mutable object graphs by accident.

Relationship Direction

Ask which object actually needs navigation.

If only `Gift` needs its `Campaign`, you may not need a `Campaign.Gifts` collection in every layer.

Unidirectional relationships can reduce complexity.

Relationship Cardinality vs Object Ownership

Cardinality answers:

EXAMPLE / FLOW

```
How many?
```

Ownership answers:

EXAMPLE / FLOW

```
Who controls lifecycle and invariants?
```

They are separate questions.

A one-to-one relationship is not automatically composition.

A one-to-many relationship is not automatically aggregation.

Composition vs Aggregation Terminology

In everyday C# development, teams often use "composition" broadly for HAS-A object construction and dependency relationships.

In UML-style terminology, aggregation and composition can be distinguished by ownership/lifetime strength.

The most important practical point is to make actual ownership and lifecycle rules explicit in code.

Composition Over Inheritance Example

Suppose you need different receipt behavior.

Poor inheritance:

EXAMPLE / FLOW

```
GiftService
    -> EmailGiftService
    -> SmsGiftService
    -> NoReceiptGiftService
```

Now receipt choice is tied to the service class hierarchy.

Composition:

EXAMPLE / FLOW

```
GiftService
    -> IReceiptSender
        -> EmailReceiptSender
        -> SmsReceiptSender
        -> NoOpReceiptSender
```

The service stays stable while the collaborator varies.

Strategy-Like Composition Preview

Without covering the full Design Patterns topic, composition often enables strategy-like behavior:

C#

```
public interface IFeePolicy
{
    decimal Calculate(decimal amount);
}
```

`GiftService` can receive an `IFeePolicy` instead of inheriting a different service subclass for each fee rule.

Decorator-Like Composition Preview

Composition can also wrap collaborators:

EXAMPLE / FLOW

```
CachingGiftRepository
    -> IGiftRepository inner
        -> SqlGiftRepository
```

The wrapper HAS-A repository and delegates while adding behavior.

This will be explored later in the Design Patterns topic.

When Inheritance Is Better

Use inheritance when:

EXAMPLE / FLOW

```
Derived type truly IS-A base type
Substitutability is natural
Shared state/behavior belongs in base
Virtual specialization is intentional
Hierarchy is stable and shallow
```

Example:

EXAMPLE / FLOW

```
GiftTransaction IS-A FinancialTransaction
```

When Composition Is Better

Use composition when:

EXAMPLE / FLOW

```
Relationship is HAS-A / USES-A
Behavior should be replaceable
You need several independent capabilities
You want to reduce base-class coupling
Ownership/collaboration is clearer than subtype identity
```

Avoid Deep Inheritance Trees

EXAMPLE / FLOW

```
A
 -> B
    -> C
       -> D
          -> E
```

A change high in the tree can affect many descendants.

Composition can often isolate changes into smaller collaborators.

Avoid God Objects

Composition does not mean one class should hold twenty unrelated services.

Bad:

```
EXAMPLE / FLOW
GiftManager
    -> repository
    -> email
    -> payment
    -> PDF
    -> audit
    -> authorization
    -> reporting
    -> logging
    -> search
    -> import
    -> export
```

Too many dependencies can indicate too many responsibilities.

Dependency Count as Design Signal

A constructor with many dependencies:

```
C#
public GiftService(
    IA a,
    IB b,
    IC c,
    ID d,
    IE e,
    IF f,
    IG g,
    IH h)
```

may reveal that the service is doing too much.

Do not hide the problem with a service locator.

Consider splitting responsibilities.

Common Mistake - Inheritance for Code Reuse

Bad reasoning:

```
EXAMPLE / FLOW
Class A already has a helper method.
I need that helper.
Therefore B inherits A.
```

Inheritance communicates subtype meaning, not merely code reuse.

Use composition or a focused helper/service when IS-A is false.

Common Mistake - Public Mutable Collections

Bad:

```C#
public List<GiftAllocation> Allocations { get; set; }
```

Callers can bypass invariants.

Expose read-only views and domain methods.

Common Mistake - Every Relationship Bidirectional

Do not add both directions automatically.

Bidirectional navigation increases coupling and cycle risk.

Add a direction only when the domain/application genuinely needs it.

Common Mistake - Treating Aggregation as a C# Keyword

C# has no `aggregation` keyword.

Aggregation is a modeling concept inferred from references, ownership, and lifecycle.

Common Mistake - Assuming Composition Requires new Inside the Owner

This is composition:

```C#
public GiftService(IGiftRepository repository)
{
    _repository = repository;
}
```

The dependency is supplied from outside, but the service still HAS-A repository collaborator.

Composition does not require internal construction.

Common Mistake - Hidden Dependencies

Bad:

```C#
var client = new HttpClient();
var repository = new SqlGiftRepository();
```

inside business methods.

Required collaborators should usually be explicit.

Common Mistake - Entity Graph Equals API Contract

Do not return an entire tracked EF Core object graph just because navigation properties exist.

API contracts should be deliberate DTOs/projections.

This reduces coupling, cycles, over-posting, and accidental data exposure.

Common Mistake - Cross-Tenant Relationships

In a multi-tenant system, object relationships must preserve tenant boundaries.

A gift from Tenant A should not be associated with a campaign or fund from Tenant B.

Relationship methods or services should validate tenant consistency where appropriate.

Security Considerations

Relationships can expose more data than intended.

Example:

EXAMPLE / FLOW

```
Gift -> Constituent -> Addresses -> PII
```

If an API serializes the entire graph, it may leak unnecessary information.

Prefer narrow DTOs and explicit projections.

Security - Tenant Isolation

Before connecting objects:

C#

```
if (gift.TenantId != fund.TenantId)
{
    throw new InvalidOperationException("Cross-tenant allocation is not allowed.");
}
```

The exact enforcement point depends on architecture, but tenant boundaries must not be implicit assumptions.

Security - Authorization and Relationship Mutation

A user being able to edit a gift does not automatically mean they can:

EXAMPLE / FLOW

```
Move it to another constituent
Change campaign ownership
Reallocate to any fund
Replace audit owner
```

Authorization should protect sensitive relationship mutations.

Performance Considerations

Large object graphs can create:

EXAMPLE / FLOW

```
More allocations
More database joins
More lazy/eager loading
More serialization work
N+1 query risks
Higher memory retention
```

Load only the relationships needed for the use case.

EF Core Awareness

The object relationship model often maps to database relationships, but do not let persistence convenience completely dictate domain design.

Examples:

EXAMPLE / FLOW

```
Constituent 1:M Gift
Campaign 1:M Gift
Gift 1:M GiftAllocation
Gift M:N Fund through GiftAllocation
Constituent M:N Address through ConstituentAddress
```

Use explicit link entities when relationship data has meaning.

Avoid N+1 Relationship Traversal

Code such as:

EXAMPLE / FLOW

```
For each constituent
    load gifts
    for each gift
        load allocations
```

can become expensive if each navigation triggers a separate database query.

Shape queries deliberately in data-access code.

Relationship Collections and Memory

Holding a large aggregate graph in memory can be expensive.

Do not load thousands of child objects merely because a collection navigation exists.

Use paging, projections, and targeted queries when appropriate.

ASP.NET Core Composition Root

Application startup is where dependency relationships are commonly registered:

C#

```
builder.Services.AddScoped<IGiftRepository, SqlGiftRepository>();
builder.Services.AddScoped<IGiftReceiptSender, EmailGiftReceiptSender>();
builder.Services.AddScoped<IGiftService, GiftService>();
```

The DI container composes the runtime graph.

Constructor Injection Example

C#

```
public sealed class GiftService : IGiftService
{
    private readonly IGiftRepository _repository;
    private readonly IGiftReceiptSender _receiptSender;

    public GiftService(
        IGiftRepository repository,
        IGiftReceiptSender receiptSender)
    {
        _repository = repository;
        _receiptSender = receiptSender;
    }
}
```

Dependencies are visible and testable.

Test Composition

Production graph:

EXAMPLE / FLOW

```
GiftService
    -> SqlGiftRepository
    -> EmailGiftReceiptSender
```

Test graph:

EXAMPLE / FLOW

```
GiftService
    -> FakeGiftRepository
    -> FakeGiftReceiptSender
```

The service logic can be tested without SQL Server or email infrastructure.

Complete Fundraising Aggregate Example

C#

```
public sealed class Gift
{
    private readonly List<GiftAllocation> _allocations = [];

    public Gift(long giftId, long tenantId, long constituentId, decimal totalAmount)
    {
        if (giftId <= 0) throw new ArgumentOutOfRangeException(nameof(giftId));
        if (tenantId <= 0) throw new ArgumentOutOfRangeException(nameof(tenantId));
        if (constituentId <= 0) throw new ArgumentOutOfRangeException(nameof(constituentId));
        if (totalAmount <= 0m) throw new ArgumentOutOfRangeException(nameof(totalAmount));

        GiftId = giftId;
        TenantId = tenantId;
        ConstituentId = constituentId;
        TotalAmount = totalAmount;
    }

    public long GiftId { get; }
    public long TenantId { get; }
    public long ConstituentId { get; }
    public decimal TotalAmount { get; }
    public Campaign? Campaign { get; private set; }
    public IReadOnlyList<GiftAllocation> Allocations => _allocations;

    public void AssignCampaign(Campaign campaign)
    {
        ArgumentNullException.ThrowIfNull(campaign);

        if (campaign.TenantId != TenantId)
        {
            throw new InvalidOperationException("Campaign must belong to the same tenant.");
        }

        Campaign = campaign;
    }

    public void AllocateTo(Fund fund, decimal amount)
    {
```

```
        ArgumentNullException.ThrowIfNull(fund);

        if (fund.TenantId != TenantId)
        {
            throw new InvalidOperationException("Fund must belong to the same tenant.");
        }

        if (amount <= 0m)
        {
            throw new ArgumentOutOfRangeException(nameof(amount));
        }

        decimal allocated = _allocations.Sum(x => x.Amount);

        if (allocated + amount > TotalAmount)
        {
            throw new InvalidOperationException("Allocations cannot exceed gift total.");
        }

        _allocations.Add(new GiftAllocation(GiftId, fund.FundId, amount));
    }
}
```

GiftAllocation

C#

```
public sealed class GiftAllocation
{
    internal GiftAllocation(long giftId, long fundId, decimal amount)
    {
        GiftId = giftId;
        FundId = fundId;
        Amount = amount;
    }

    public long GiftId { get; }
    public long FundId { get; }
    public decimal Amount { get; }
}
```

The gift owns the mutation pathway for allocations in this example.

Campaign

C#

```
public sealed class Campaign
{
    public Campaign(long campaignId, long tenantId, string name)
    {
        CampaignId = campaignId;
        TenantId = tenantId;
        Name = name;
    }

    public long CampaignId { get; }
    public long TenantId { get; }
    public string Name { get; }
}
```

A campaign is referenced by gifts but is not owned by an individual gift.

Fund

C#

```
public sealed class Fund
{
    public Fund(long fundId, long tenantId, string name)
    {
        FundId = fundId;
        TenantId = tenantId;
        Name = name;
    }

    public long FundId { get; }
    public long TenantId { get; }
    public string Name { get; }
}
```

The fund has an independent lifecycle.

Complete Relationship Flow

EXAMPLE / FLOW

```
Constituent
    -> Gift
        -> Campaign (association)
        -> GiftAllocation (strong owned relationship)
            -> Fund (association)
```

Service-level collaboration:

EXAMPLE / FLOW

```
GiftsController
    -> IGiftService
        -> GiftService
            -> IGiftRepository
            -> IGiftReceiptSender
            -> IOutboxWriter
```

Relationship Decision Flow

EXAMPLE / FLOW

```
Does B truly represent a specialized kind of A?
    ↓
Yes -> IS-A -> consider inheritance
No  -> continue
```

EXAMPLE / FLOW

```
Does A need/use/own B?
    ↓
Yes -> HAS-A / USES-A
    -> association / aggregation / composition / dependency
```

Then ask:

EXAMPLE / FLOW

```
Can B exist independently?
    -> likely association/aggregation
```

```
Does A control B's lifecycle/invariants?
    -> strong composition candidate
```


Practical Exercise 1 - Identify Relationships

Classify these:

EXAMPLE / FLOW

```
Gift -> Campaign
Gift -> GiftAllocation
GiftAllocation -> Fund
GiftService -> IGiftRepository
GiftTransaction -> FinancialTransaction
```

Explain whether each is IS-A, HAS-A, USES-A, association, composition, or inheritance.

Practical Exercise 2 - Encapsulate a Collection

Start with:

C#

```
public List<GiftAllocation> Allocations { get; set; } = [];
```

Refactor to:

- private backing collection;
- read-only exposure;
- controlled `AllocateTo` method;
- validation that allocations do not exceed gift amount.

Practical Exercise 3 - Replace Inheritance with Composition

Start with:

EXAMPLE / FLOW

```
EmailGiftService : GiftService
SmsGiftService : GiftService
```

Refactor to:

EXAMPLE / FLOW

```
GiftService -> IReceiptSender
```

with multiple receipt sender implementations.

Practical Exercise 4 - Many-to-Many Link Entity

Model:

EXAMPLE / FLOW

```
Gift M:N Fund
```

using:

containing `GiftId`, `FundId`, and `Amount`.

Practical Exercise 5 - Tenant Safety

Add a rule preventing a gift from being assigned to a campaign or fund from another tenant.

Practical Exercise 6 - DI Composition

Create a `GiftService` that depends on:

EXAMPLE / FLOW

```
IGiftRepository
IGiftReceiptSender
IClock
```

Then create production and test graphs.

Interview Questions

1. What is association?

A general relationship where one object knows about, references, or interacts with another object.

2. What is aggregation?

A whole-part relationship where the parts can usually exist independently of the whole.

3. What is composition?

A strong HAS-A relationship where an object is built from collaborators/parts and often controls or strongly defines their use, invariants, or lifecycle.

4. Does C# have an aggregation keyword?

No. Aggregation is a modeling concept represented through references and lifecycle rules.

5. What is a HAS-A relationship?

One object contains, owns, or depends on another object/capability.

6. What is an IS-A relationship?

A subtype relationship where the derived type is genuinely a specialized kind of the base type.

7. What is a dependency relationship?

One component requires another component to perform work.

8. What is object collaboration?

Several objects coordinate by calling one another to complete a larger use case while each retains focused responsibilities.

9. What is composition over inheritance?

Prefer assembling behavior from collaborators when the relationship is HAS-A or when inheritance would be used only for code reuse.

10. Why is composition often preferred over inheritance?

It can reduce base-class coupling, avoid deep hierarchies, support multiple independent capabilities, and make behavior easier to replace/test.

11. Is inheritance always worse than composition?

No. Inheritance is appropriate for genuine IS-A subtype relationships with substitutability.

12. What is delegation?

An object forwards part of its behavior to a collaborator it contains or references.

13. What is constructor injection?

Required dependencies are provided through the constructor and stored for later use.

14. Why are explicit dependencies better than hidden dependencies?

They make requirements visible, improve testability, and reduce hard-coded infrastructure coupling.

15. What is a one-to-many relationship?

One object/entity is related to multiple objects/entities, such as `Constituent -> Gifts`.

16. What is a many-to-many relationship?

Multiple objects on each side can relate to multiple objects on the other side, often through a link entity.

17. Why use a link entity such as GiftAllocation?

Because the relationship itself has data such as allocation amount and may have its own rules.

18. Is cardinality the same as ownership?

No. Cardinality says how many; ownership says who controls lifecycle and invariants.

19. Why can public mutable collections be dangerous?

Callers can bypass domain rules and create inconsistent relationship state.

20. How should relationship collections be exposed?

Often through read-only interfaces with controlled methods for mutation when invariants matter.

21. What is a bidirectional relationship?

Both objects can navigate to each other.

22. Why can bidirectional relationships be problematic?

They can create synchronization complexity, serialization cycles, tighter coupling, and larger graphs.

23. What is a cyclic object graph?

A graph where references eventually point back to an earlier object, such as `Campaign -> Gift -> Campaign`.

24. How can cyclic graphs affect APIs?

Direct serialization can loop, duplicate data, or expose too much information. DTOs/projections are usually safer.

25. How does dependency injection relate to composition?

A DI container assembles composed object graphs by creating objects and supplying their dependencies.

26. Does using DI automatically mean good composition?

No. A class can still have too many dependencies or poor responsibilities.

27. What is the difference between composition and aggregation?

Composition usually implies stronger ownership/lifetime control; aggregation describes a weaker whole-part relationship where parts can live independently.

28. Give a Fundraising example of association.

A `Gift` referencing an optional `Campaign`, where the campaign has an independent lifecycle.

29. Give a Fundraising example of strong composition.

A `Gift` controlling creation and mutation of its `GiftAllocation` collection can be modeled as a strong owned relationship.

30. Why is GiftAllocation not just a List?

Because the gift-fund relationship carries data such as allocation amount and may require its own rules.

31. What is a hidden dependency?

A dependency created or located inside implementation code rather than declared in the public construction/operation contract.

32. Why can service locator be problematic?

It hides dependencies behind a generic provider and makes design/test requirements less explicit.

33. How can composition improve testing?

Production collaborators can be replaced with fakes while the object under test remains unchanged.

34. What is the fragile base-class problem?

Changes in a base class can unexpectedly affect derived classes that depend on its extension points or implementation details.

35. How does composition reduce fragile-base risks?

Behavior is isolated behind collaborators instead of inherited deeply through a class hierarchy.

36. Can composition still create tight coupling?

Yes, especially if a class depends directly on many concrete implementations or exposes their internals.

37. Why use interfaces with composition?

Interfaces can make collaborators replaceable and allow callers to depend on capabilities rather than technologies.

38. What is relationship encapsulation?

Only controlled methods are allowed to create/change a relationship so invariants remain valid.

39. Why validate tenant IDs across relationships?

To prevent cross-tenant data links that could violate isolation and security boundaries.

40. What is the main design question before choosing inheritance?

Ask whether the derived type is truly substitutable as a specialized version of the base type. If the real relationship is HAS-A, use composition/association instead.

Best Practices

- Model IS-A with inheritance only when subtype semantics are real.
- Model HAS-A/USES-A with composition, association, or dependencies.
- Prefer composition when the goal is behavior reuse rather than subtype identity.
- Keep object graphs as small and directional as the use case requires.
- Avoid making every relationship bidirectional.
- Encapsulate collection mutation when business invariants matter.
- Use link entities when relationships carry their own data.
- Make required dependencies explicit through constructors.
- Depend on abstractions when meaningful substitution exists.
- Avoid service locator for normal required dependencies.
- Keep services focused; too many dependencies can indicate too many responsibilities.
- Protect tenant boundaries when connecting domain objects.
- Avoid returning rich entity graphs directly from APIs.
- Use DTOs/projections to control serialization and data exposure.
- Load only the relationships needed for a query/use case.
- Do not let persistence cardinality automatically dictate domain ownership semantics.
- Use composition and polymorphism together for flexible replaceable behavior.

Quick Revision

Concept	Meaning
Association	General connection between objects
Aggregation	Whole-part, part can live independently
Composition	Strong HAS-A / assembled behavior / often strong ownership
Dependency	One object requires another to do work
Collaboration	Objects coordinate to complete a use case

Concept	Meaning
IS-A	Inheritance relationship
HAS-A	Composition/association relationship
USES-A	Dependency relationship
Delegation	Forward behavior to collaborator
Constructor injection	Supply required dependency at construction
Link entity	Object representing relationship plus relationship data
Bidirectional relationship	Both sides navigate each other
Composition over inheritance	Prefer collaborators when subtype identity is not real

Final End-to-End Concept Flow

EXAMPLE / FLOW

```
Need relationship
    ↓
Is one type truly a specialized kind of another?
    ↓ yes
Inheritance / IS-A
```

Otherwise:

EXAMPLE / FLOW

```
Does one object know/use/reference another?
    ↓
Association / Dependency
```

If it forms a whole-part relationship:

EXAMPLE / FLOW

```
Can part live independently?
    -> yes: aggregation-like relationship
    -> no / owner controls lifecycle & invariants: composition-like relationship
```

Fundraising model:

EXAMPLE / FLOW

```
Constituent
    -> Gifts

Gift
    -> Campaign
    -> GiftAllocations

GiftAllocation
    -> Fund
```

Application composition:

EXAMPLE / FLOW

```
GiftsController
    -> IGiftService
        -> GiftService
            -> IGiftRepository
```

```
        -> IGiftReceiptSender
        -> IOutboxWriter
```


The core lesson is:

> **Inheritance models IS-A specialization. Composition, association, and dependencies model HAS-A and USES-A collaboration. Prefer composition when you only need reusable or replaceable behavior, keep ownership and lifetime rules explicit, and encapsulate relationship changes so the object graph cannot bypass business invariants.**

Next roadmap topic: Topic 17 - Structs, Records and Enums.

References

- Microsoft Learn - Inheritance and polymorphism in C#.
- Microsoft Learn - Dependency injection in .NET / ASP.NET Core.
- Microsoft Learn - EF Core relationship modeling guidance.